SENTINEL TRADER

Master Project Specification, Implementation Blueprint and Three-Person Collaboration Plan

Project type
Multi-model quantitative trading and portfolio-management agent

Core objective
Maximize risk-adjusted portfolio performance under realistic market friction, execution constraints and portfolio risk limits.

Integrated friction-aware strategy selection • OHLCV intelligence • Financial news • Foundation models • Risk management • Profit realization

Team size: 3 collaborators

Development environment: Python, Google Colab, Hugging Face, GitHub and the competition's actual trading interface.

Version 1.0 • Working specification • Adapt to the official competition API and rules before implementation.

PART I: COMPLETE PROJECT DEFINITION

1. Project Vision

Sentinel Trader is a regime-aware, multi-model trading and portfolio-management agent that analyzes OHLCV data, financial news, market conditions, execution costs and current portfolio state to choose among long, short, hold, reduce, exit and other supported actions.

Unlike a conventional trading bot that buys when a moving average crosses another moving average, Sentinel evaluates whether a strategy is appropriate for the current market, whether its expected return survives transaction costs, and whether the resulting position improves the portfolio as a whole.

- Quantitative OHLCV feature engineering.
- Trend-following and moving-average crossover strategies.
- Mean-reversion strategies.
- Financial news sentiment and price-reaction analysis.
- Pretrained time-series foundation models.
- A custom machine-learning model trained on historical market features.
- Market-regime detection.
- Friction-aware strategy selection and trade evaluation.
- Portfolio-level risk and position management.
- Unrealized-profit management and capital recycling.
- Event-driven historical replay and rigorous evaluation.

2. Central Research Question

Can a regime-aware, multi-model trading system outperform simpler strategies by selecting actions based on expected net edge, uncertainty, execution friction and portfolio state rather than price direction alone?

The system must demonstrate this using controlled experiments, chronological testing and realistic trading costs. A complicated architecture is not evidence of profitability.

3. System Design Principles

- Evidence before complexity: Every model must demonstrate incremental value against a simpler baseline.
- No hardcoded market assumptions: Tickers, thresholds, trading windows, model paths, strategy parameters, costs, risk limits and competition-specific rules must be configuration-driven or derived from data and documented rules.
- No future-data leakage: Every decision must use only information available at that timestamp.
- Friction-aware by construction: Every candidate trade is evaluated after expected costs.
- Portfolio-aware decisions: A profitable individual trade may still be rejected if it worsens portfolio risk.
- Risk enforcement is deterministic: ML models cannot bypass cash, margin, exposure, data-validity or order-size constraints.
- Actual execution state is authoritative: Accounting reflects accepted orders and actual fills, not model intentions.
- Reproducible evaluation: Model versions, configurations, datasets and evaluation results are traceable.
- Graceful degradation: If a nonessential model fails, the system follows a documented fallback policy.
- Competition compatibility: The official simulator, supported instruments, execution API, scoring rules and time budget take precedence.

PART II: COMPLETE SYSTEM ARCHITECTURE

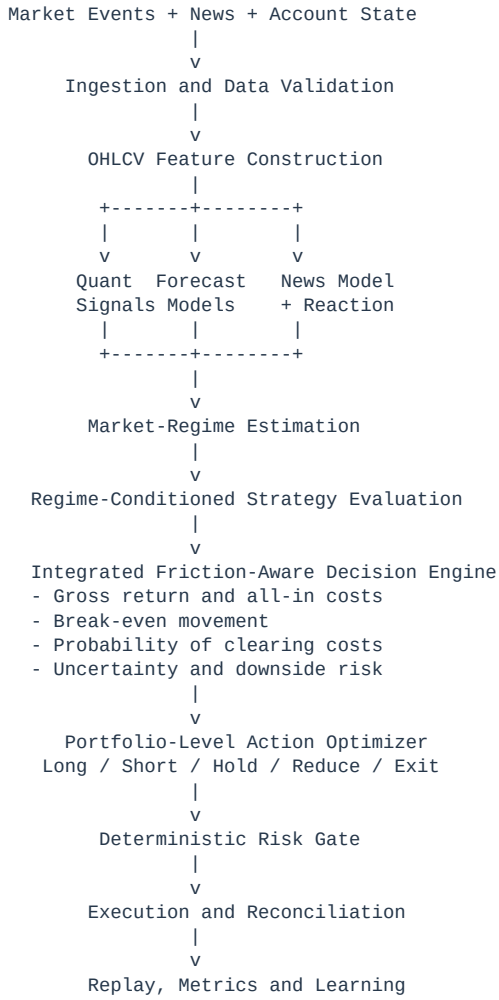
4. End-to-End Workflow

1. Event ingestion: Receive market events, OHLCV bars, available news, account state, order updates and execution reports. Record timestamps and validate schemas.
2. Market-state construction: Calculate OHLCV features, technical signals, volume characteristics, volatility, benchmark-relative movement and data-quality indicators. Build a timestamp-consistent state of positions,

cash, exposure and unrealized P&L.

3. Model inference: Run quantitative strategies, a custom OHLCV model, an optional validated time-series model, a financial-news sentiment model and a market-regime classifier. Each returns standardized outputs with uncertainty, timestamp and version.
4. Regime-conditioned strategy evaluation: Estimate the suitability and reliability of trend following, mean reversion, news-driven trading and other validated strategies under the current regime.
5. Integrated friction-aware decision evaluation: Generate feasible actions and estimate expected gross return, all-in execution costs, uncertainty, risk contribution and opportunity cost. Jointly consider strategy, entry, holding, reduction, exit, order type and size.
6. Portfolio utility and capital allocation: Rank feasible actions by expected portfolio-level utility while considering exposure, concentration, correlation, cash, margin and the value of preserving capital.
7. Deterministic risk gate: Reject or constrain actions that violate risk configuration, supported trading rules, data validity or execution constraints.
8. Execution: Submit supported orders through the actual interface and handle acknowledgements, partial fills, rejections, cancellations and timeouts.
9. State reconciliation: Update positions, cash, realized P&L, unrealized P&L, costs and risk exposure using authoritative execution records.
10. Outcome tracking: Record decision outcomes, model performance, regime-specific results, execution quality and alternative-policy performance in replay.

5. Architecture Flow



PART III: MARKET ANALYSIS AND MODEL STRATEGY

6. OHLCV Analysis

OHLCV data is the primary quantitative foundation. The feature pipeline must be causal, reproducible and explicit about what the data can and cannot reveal.

6.1 Feature families

- Price structure: Returns over configurable windows, candle body and wick characteristics, gaps, swing structure and distance from rolling reference prices.
- Trend: Fast and slow moving averages, slopes, crossover direction and recency, and trend-strength measures such as ADX where available.
- Mean reversion: Rolling price z-scores, deviations from rolling means or valid VWAP, RSI and validated oscillators.
- Volatility and downside risk: ATR, rolling realized volatility, volatility changes, downside-return statistics and gap risk.
- Volume and liquidity proxies: Relative volume, price-volume divergence, rolling traded volume and order size relative to available or estimated liquidity.
- Market context: Benchmark-relative returns and cross-asset correlation where supported.
- Data integrity: Missing or duplicated bars, stale timestamps, invalid OHLC relationships, abnormal jumps and corporate-action adjustments where applicable.

Rolling calculations must not incorporate future bars. Any normalization or imputation learned from data must be fitted on training data only. OHLCV alone cannot reveal exact order-book depth, queue priority or precise intrabar paths; document execution assumptions.

7. Trading Strategies

7.1 Trend following

Use moving-average crossover, trend slope, momentum and trend-strength information to estimate opportunities. A crossover is a signal, not an unconditional order. Evaluate regime, signal strength, expected return, horizon, break-even movement and portfolio risk.

7.2 Mean reversion

Use rolling deviations, oscillators and volatility-adjusted dislocations to identify possible reversals. Restrict or downweight the strategy in persistent trends or stress regimes inconsistent with its assumptions.

7.3 News-driven trading

Derive sentiment probabilities, novelty, event category where supported, publication and availability timestamps, and abnormal price/volume reaction. Distinguish positive sentiment from positive expected trading return. Every news signal must pass the same net-edge and risk evaluation as other strategies.

8. Model Selection and Training

8.1 Custom OHLCV model

Begin with LightGBM or XGBoost. Targets may include forward returns, probability of positive return after costs, target-before-stop outcomes or conditional downside/volatility. The target must match the decision being made.

8.2 Pretrained time-series model

Evaluate a candidate such as Amazon Chronos or Google's TimesFM after verifying current availability, license, runtime, input requirements and compatibility. Compare forecast error, directional accuracy, calibration, runtime, memory and incremental trading value against simple baselines.

8.3 Financial sentiment model

Evaluate FinBERT or another financial-domain model. Process timestamped news and expose probabilities or scores with documented meaning. The sentiment model must never submit orders directly.

8.4 Market-regime model

Start with a transparent baseline using volatility and trend strength. Compare with an HMM, clustering approach or temporal classifier only if validation supports the extra complexity. Produce regime probabilities or scores.

8.5 State-action model

Estimate outcomes of feasible actions given market and portfolio state. Start with supervised action-outcome prediction and a constrained utility-based policy. Consider reinforcement learning only after accounting, execution and simulator behavior are validated.

PART IV: FRICTION-AWARE DECISION-MAKING

9. Integrated Friction-Aware Strategy Selection

Friction-aware selection is a core capability, not an optional add-on. The system evaluates the cost of acting before choosing the strategy, action, order type and quantity.

9.1 Break-even movement

```
Break-even movement
  = Entry costs + Expected exit costs + Other applicable costs
```

Express costs relative to position notional or quantity as appropriate. For shorts, include covering, financing and other applicable costs. Match the competition's fee schedule, price convention, order mechanics and accounting rules. Do not use one universal friction percentage for every asset and order.

9.2 Cost model

- Bid-ask spread.
- Commission and transaction fees.
- Slippage.
- Market impact.
- Latency-related price movement.
- Borrow or funding costs for supported short positions.
- Taxes where required by competition rules.
- Order rejection, partial-fill and cancellation effects where measurable.

Each cost estimate must identify its source, units, timestamp and method. Where exact values are unavailable, use documented configurable assumptions and stress tests rather than presenting estimates as observed facts.

9.3 Probability of clearing costs

```
P(Gross return > All-in trading costs)
```

Estimate this probability conditioned on action, asset, quantity, horizon and available information. Consider both expected net return and the probability that gross payoff exceeds costs. A high probability of a small gain does not guarantee positive expected value.

9.4 Strategy and execution selection

- Prefer a more liquid opportunity over a less liquid one when net utility supports it.
- Compare trend following with mean reversion under the current regime.
- Compare rapid crossover with lower-turnover alternatives.
- Compare holding an existing position with closing and reopening it.
- Consider a supported limit order versus an aggressive order only when the fill model is realistic.
- Wait for a better cost-to-edge ratio when appropriate.
- Choose no trade when no candidate has sufficient net edge.

A limit order must not be assumed to fill just because its price is attractive. If the simulator lacks realistic limit-order mechanics, do not claim a fill advantage that cannot be evaluated.

9.5 Friction-aware utility

```
Expected net return
  = Expected gross return - Expected all-in costs

Utility(action)
  = Expected portfolio return
    - Risk penalty
    - Forecast uncertainty penalty
    - Applicable execution and opportunity costs
```

Use a documented, configurable objective. Estimate the effect on the whole portfolio where possible instead of treating trades independently.

9.6 Mandatory abstention

NO_TRADE is a valid action when expected net edge is insufficient, uncertainty is excessive, costs overwhelm the payoff, data is invalid or stale, constraints prohibit the action, a model fails without a valid fallback, or no candidate clears the configured threshold.

PART V: PORTFOLIO RISK AND PROFIT REALIZATION

10. Risk Management

Risk enforcement is independent of predictive models. Validate available cash and buying power, position size, gross/net exposure, concentration and correlation, short availability and margin, drawdown responses, order validity, data freshness, duplicate orders and current execution state. Limits must come from validated configuration or official rules. No model can override them.

11. Realized and Unrealized P&L

11.1 Accounting

```
Long unrealized P&L
  = (Current mark price - Cost basis per unit)
    * Open quantity
```

For shorts, reverse the price direction and include applicable borrow, margin and covering costs. Realized P&L is recorded as positions are closed under the competition's cost-basis and fee rules.

11.2 Profit-realization policy

Choose between hold, reduce, partial exit and full exit based on remaining expected upside, uncertainty, reversal risk, current P&L, exit cost, portfolio exposure, alternative opportunities and applicable tax rules. Do not use a fixed profit percentage as the only exit rule.

11.3 Partial exits

1. Calculate the allowed exit quantity.
2. Submit the order using the supported interface.
3. Reconcile actual filled quantity.
4. Calculate realized P&L and applicable costs on filled quantity.
5. Update remaining open quantity and unrealized P&L.
6. Recalculate cash, exposure and risk limits.
7. Reevaluate other opportunities.

11.4 Capital recycling

After an exit, realized cash becomes available subject to settlement, buying-power and competition rules. Compare new opportunities by expected net utility, risk and capital requirements. Do not automatically reinvest every realized gain.

11.5 Evaluation

Compare hold-until-signal-reversal, fixed take-profit, trailing-stop, fixed partial-exit and learned partial-exit/capital-recycling policies using identical capital, data, execution assumptions and risk constraints. Measure wealth, drawdown, realized P&L, profit giveback, turnover, fees and missed upside.

PART VI: IMPLEMENTATION AND REPOSITORY DESIGN

12. Proposed Repository Structure

Inspect the existing competition repository first. Preserve mandatory entrypoints and required files. Adapt this structure rather than rebuilding working infrastructure without reason.

```
sentinel-trader/
■■■ README.md
■■■ pyproject.toml
■■■ configs/
■   ■■■ base.yaml
■   ■■■ development.yaml
■   ■■■ evaluation.yaml
■   ■■■ competition.yaml
■■■ schemas/
■   ■■■ market_event.schema.json
■   ■■■ model_prediction.schema.json
■   ■■■ trade_decision.schema.json
■   ■■■ execution_event.schema.json
■■■ src/sentinel/
■   ■■■ ingestion/
■   ■■■ features/
■   ■■■ models/
■   ■■■ strategies/
■   ■■■ regime/
■   ■■■ decision/
■   ■■■ friction/
■   ■■■ portfolio/
■   ■■■ risk/
■   ■■■ execution/
■   ■■■ accounting/
■   ■■■ evaluation/
■   ■■■ observability/
■■■ artifacts/README.md
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ replay/
■■■ notebooks/README.md
■■■ scripts/
■■■ reports/
■■■ docs/
    ■■■ architecture.md
    ■■■ interfaces.md
    ■■■ evaluation_protocol.md
    ■■■ decisions.md
```

13. Configuration and No-Hardcoding Policy

- Configure instrument universe, lookback windows, feature settings, model IDs/paths, model enablement, decision horizons and strategy parameters.
- Configure cost schedules, risk limits, exposure caps, uncertainty handling, supported order types, data freshness, evaluation windows and seeds.
- Do not embed ticker names, sample account IDs, competition-specific fees, fixed risk limits, evaluation answers or dataset-specific outcomes in business logic.
- Do not assume all instruments support short selling or limit orders.
- Do not use hardcoded timestamps, future labels or evaluation answers in features.
- Do not hide missing data or model failures by fabricating defaults.
- Keep secrets in environment variables or the approved secret mechanism; never commit them.
- Validate configuration at startup and fail clearly when required settings are absent or invalid.
- Deterministic safety checks may be implemented in code, but competition-specific limits must be configured or derived from authoritative rules.
- Verify the official execution interface and constraints from source rather than guessing.

PART VII: THREE-PERSON COLLABORATION PLAN

14. Team Ownership Rules

Person 1 owns market data, execution and accounting. Person 2 owns features, ML models and regime estimation. Person 3 owns strategy selection, friction-aware decisions, portfolio risk and evaluation. Teammates may write integration tests for other subsystems, but only the assigned owner changes that subsystem's production implementation without coordination.

15. Person 1: Infrastructure, Market Data, Execution and Accounting

Exclusive production ownership

- `src/sentinel/ingestion/`
- `src/sentinel/execution/`
- `src/sentinel/accounting/`
- `src/sentinel/observability/`
- `schemas/market_event.schema.json`
- `schemas/execution_event.schema.json`
- Ingestion, execution and accounting integration tests.

Deliverables

- Market ingestion: Parse actual input format, validate timestamps and schemas, handle duplicates, missing fields, out-of-order events and stale data.
- Execution adapter: Wrap official API, submit supported orders, handle acknowledgements, rejections, cancellations, partial fills and timeouts; reconcile uncertain order states before retrying.
- Accounting: Maintain cash, positions, cost basis, realized/unrealized P&L, fees and supported long/short accounting; reconcile with authoritative simulator state.
- Reliability: Structured logs, health signals, startup/shutdown and recovery behavior.

Must not implement: predictive models, strategy weights or the final trade-selection policy.

Completion criteria

- Market events parse and validate correctly.
- Invalid and stale events are explicitly handled.
- Execution and accounting agree after complete, partial and rejected orders.
- Integration tests pass against the actual interface or a faithful test double.
- The required competition entrypoint remains compatible.

16. Person 2: Quantitative Features, Pretrained Models and Market Regimes

Exclusive production ownership

- `src/sentinel/features/`
- `src/sentinel/models/`
- `src/sentinel/regime/`
- `schemas/model_prediction.schema.json`
- `notebooks/` and model training/validation scripts
- `artifacts/README.md`
- Model-specific unit tests and evaluation reports.

Deliverables

- Build causal trend, crossover, mean-reversion, volatility, volume and benchmark-relative features.
- Train a custom OHLCV baseline with chronological training and validation; document targets, features, cutoff and version.
- Evaluate a time-series foundation model such as Chronos or TimesFM, including runtime, memory and incremental value.
- Evaluate FinBERT or another financial-domain model with timestamp checks, deduplication and missing-news handling.
- Build a transparent regime baseline and compare advanced alternatives only if justified.
- Report out-of-sample performance, calibration, regime-specific results and models that do not add value.

Must not implement: production execution, accounting or final trade authorization.

Completion criteria

- Feature pipeline is deterministic for a given input/configuration.
- Models load from configured artifacts.
- Missing artifacts produce explicit errors or documented fallback outputs.
- Predictions include timestamps, versions and defined score semantics.
- Chronological validation and leakage checks pass.
- Inference meets the agreed latency budget.

17. Person 3: Decision Intelligence, Friction, Risk and Evaluation

Exclusive production ownership

- `src/sentinel/strategies/`
- `src/sentinel/decision/`
- `src/sentinel/friction/`
- `src/sentinel/portfolio/`
- `src/sentinel/risk/`
- `src/sentinel/evaluation/`
- `schemas/trade_decision.schema.json`
- Decision, friction, risk and replay tests; evaluation reports.

Deliverables

- Define trend and mean-reversion signals using Person 2's features and integrate news/regime outputs without duplicating model code.
- Estimate break-even movement and all-in costs; estimate probability of clearing costs when sufficient data exists.
- Evaluate actions across relevant order sizes and horizons, including trade, hold, reduce, exit and abstain.
- Score expected net utility, uncertainty and portfolio risk; enforce configured cash, exposure, margin and data-quality constraints.
- Implement profit realization, partial exits and capital recycling using actual fills and accounting outputs.
- Compare against baselines and run ablations for models, regimes, friction and exit policies.

Must not implement: internal execution adapter or model-training pipeline.

Completion criteria

- Every candidate action is evaluated with appropriate friction.
- Risk violations are rejected and no-trade is valid.
- Accounting is reconciled after fills.
- Evaluation is reproducible and includes realistic baselines.

PART VIII: FROZEN INTERFACES TO PREVENT COLLISIONS

18. Interface Contract A: Market Events

Person 1 owns the schema and parser. Other components consume normalized events.

```
MarketEvent:
  instrument_id
  event_timestamp
  received_timestamp
  event_type
  open, high, low, close, volume
  available_quote_data
  source_id
  data_quality_status
```

Adapt fields to actual competition data. Missing fields remain explicitly unavailable instead of being invented.

19. Interface Contract B: Model Predictions

Person 2 owns the model-output contract.

```
ModelPrediction:
  model_id
  model_version
  prediction_timestamp
  input_cutoff_timestamp
  instrument_id
  prediction_type
  horizon
  prediction_value
  prediction_distribution
  confidence_or_uncertainty
  availability_status
  metadata
```

A sentiment probability, return forecast and regime probability are different quantities. Do not treat them as interchangeable. Optional fields must be documented.

20. Interface Contract C: Trade Decisions

Person 3 owns the decision schema.

```
TradeDecision:
  decision_id
  decision_timestamp
  instrument_id
  selected_action
  target_quantity_or_exposure
  order_type
  expected_gross_return
  estimated_all_in_cost
  expected_net_return
  probability_of_clearing_costs
  uncertainty_estimate
  estimated_risk_contribution
  selected_strategy
  selection_or_rejection_reasons
  model_versions
  configuration_version
```

Unavailable estimates must be represented as unavailable, not fabricated to satisfy the schema.

21. Interface Contract D: Execution Results

Person 1 owns this contract.

```
ExecutionResult:
  order_id
  decision_id
  instrument_id
  order_status
  requested_quantity
  filled_quantity
  fill_price
  fees
  execution_timestamp
  rejection_reason
  reconciliation_status
```

Use actual simulator field names where required. The internal adapter translates external responses into the normalized format.

22. Contract Freeze Procedure

1. Person 1 proposes event and execution schemas.
2. Person 2 proposes the prediction schema.
3. Person 3 proposes the decision schema.
4. All three review the contracts together.
5. Create shared typed models or validated schemas.
6. Add representative payloads and contract tests.
7. Tag the agreed interface version in Git.

8. Any breaking change requires explicit agreement and a coordinated migration.

Once contracts are frozen, teammates can implement against mock objects independently. This prevents conflicting assumptions about field names, units, timestamps and return semantics.

PART IX: PHASED IMPLEMENTATION PLAN

23. Phase 0: Repository and Competition Audit

All three collaborate. Inspect repository, PS, execution interface, scoring rules, supported assets, long/short permissions, order types, fees, latency/runtime limits and current failures. Run the untouched baseline and agree on contracts, configuration and ownership.

Exit gate: The team understands the actual competition mechanics and can run the baseline.

24. Phase 1: Stable Foundation

Person 1 leads. Implement event validation, accounting, execution adapter, contract tests, configuration validation and startup/recovery behavior. Others use mock interfaces.

Exit gate: Events and positions reconcile without model dependencies.

25. Phase 2: Quantitative Baseline

Person 2 leads. Implement OHLCV features, crossover and mean-reversion baselines, chronological evaluation and first custom prediction model. Person 3 develops policy tests against synthetic predictions.

Exit gate: Reproducible baseline results and valid model interfaces.

26. Phase 3: Integrated Decision Engine

Person 3 leads. Implement action scoring, integrated friction-aware decisions, deterministic risk constraints, hold/reduce/partial exit/full exit/abstention and connect to mocked then real execution.

Exit gate: Every proposed order is evaluated for cost, expected value and feasibility before submission.

27. Phase 4: Foundation Models and News

Person 2 leads model validation; Person 3 integrates accepted outputs. Evaluate time-series and financial sentiment models, add regime estimation, measure incremental value and remove models that do not justify runtime or complexity.

Exit gate: Enabled models have documented inference, fallback behavior and evaluation evidence.

28. Phase 5: Profit Realization and Capital Recycling

Person 3 leads; Person 1 validates accounting. Compare exit policies, validate realized P&L after partial fills, test short accounting where supported, and measure profit giveback, final wealth and drawdown.

Exit gate: Position changes, realized P&L and available capital reconcile after supported execution outcomes.

29. Phase 6: Full Replay and Ablation

Person 3 owns evaluation. Compare crossover baseline, trend plus mean reversion, friction-aware selection, custom OHLCV model, foundation model, news/regime intelligence and full portfolio/profit policy under consistent assumptions.

Exit gate: Reproducible evidence shows which components help and how the system compares with simpler alternatives.

30. Phase 7: Competition Freeze

- Freeze validated artifacts and configuration.

- Run the actual entrypoint end to end.
- Test missing data, model failures, rejected orders, partial fills and timeouts.
- Verify latency, memory, secrets and required files.
- Prepare a concise technical explanation and measured results.
- After freeze, make only verified defect fixes; do not add an untested model at the last minute.

PART X: EVALUATION REQUIREMENTS

31. Core Metrics

Portfolio performance

- Final portfolio wealth; net return after costs; realized and unrealized P&L; maximum drawdown; downside volatility; long/short contribution; capital utilization.

Friction and execution

- Fees and transaction costs; slippage and market-impact estimates; cost as fraction of gross gains; turnover; order failures; partial-fill behavior; median and tail decision latency.

Prediction quality

- Forecast error; directional accuracy; calibration; performance by regime and horizon; incremental value over a simple baseline.

Capital management

- Realized versus unrealized P&L over time; profit giveback after peak unrealized gain; partial-exit performance; capital recycled into subsequent opportunities; performance versus holding/simpler exit policies.

Reliability

- Stale-data detection; duplicate-order prevention; accounting reconciliation; model-failure recovery; constraint-violation rate; reproducibility.

Match metric definitions to the actual competition scoring system. Win rate alone is not profitability, and a high backtest return is not evidence of robustness.

32. Required Ablations

- No-news versus news-enabled.
- Custom OHLCV model versus foundation model.
- Fixed strategy selection versus regime-conditioned selection.
- Friction-aware versus friction-unaware selection.
- Fixed exit rules versus learned profit realization.
- Full system versus strongest simpler baseline.

Use chronological holdout periods and stress-test costs. Tune on training and validation periods, not the final test period.

PART XI: TEAM WORKFLOW AND GIT RULES

33. Branch Strategy

- `work/infrastructure-execution`
- `work/quant-models`
- `work/decision-engine`

Each branch should primarily change the owner's files. Use short-lived feature branches for major changes. Avoid three people committing directly to the same integration branch.

34. Pull Request Rules

- Purpose of the change.
- Files changed.
- Interface/configuration changes.
- Tests run and actual results.
- Known limitations.
- Required migration or follow-up.

No pull request should silently alter a frozen interface.

35. Shared Files

Coordinate changes to `pyproject.toml`, shared configuration schemas, type definitions, competition entrypoint, dependency lock files, CI/test configuration and shared interface documentation. Assign one integration owner per shared file.

36. Integration Schedule

1. Each teammate pushes their branch at the end of each phase.
2. Run subsystem tests.
3. Merge only contract-compatible changes.
4. Run integration tests.
5. Record the working commit and configuration version.
6. Update phase status in project documentation.

Do not postpone integration until the last day.

PART XII: FIRST MASTER IMPLEMENTATION PROMPT

37. Copy-Paste Prompt for the Coding Agent

Use this prompt after placing the document in the repository as `docs/master_project_spec.md`. The agent must audit existing code before implementation.

```
You are the lead software architect and implementation engineer for Sentinel Trader, a multi-model, regime-aware trading and

Read docs/master_project_spec.md completely before changing code.

MANDATORY FIRST STEP
Inspect the repository, official competition interfaces, execution contract, configuration, dependencies, current entrypoint,

Produce a read-only audit identifying:
- Current architecture and working functionality.
- Required competition interfaces.
- Existing hardcoded assumptions.
- Data leakage risks.
- Execution and accounting risks.
- Missing modules.
- Dependencies and latency constraints.
- Recommended implementation sequence.

Do not overwrite or restructure the project before this audit.

IMPLEMENTATION RULES
1. Implement one phase at a time.
2. Preserve required competition entrypoints and API contracts.
3. Do not hardcode tickers, competition-specific fees, risk limits, sample answers, timestamps, credentials or model outputs.
4. Load configurable values from validated configuration or authoritative runtime data.
5. Never use future information in live decisions or historical features.
6. Do not invent fields or assume unsupported order types.
7. Do not fabricate costs, fills, predictions or evaluation results.
8. Keep forecasting, action selection, risk enforcement, execution and accounting separate through explicit interfaces.
9. Integrate friction-aware strategy selection into the core decision pipeline, not as a disconnected add-on.
10. Make NO_TRADE a valid action.
```

```
11. Reconcile positions and P&L using actual fills and competition accounting rules.
12. Treat pretrained models as optional until value, runtime and fallback behavior are validated.
13. Keep inference within competition latency and runtime limits.
14. Use typed contracts, tests and reproducible replay evaluation.
15. Do not implement reinforcement learning until the simulator, accounting and baseline policy are validated.

COLLABORATION RULES
- Person 1: ingestion, execution, accounting and observability.
- Person 2: features, pretrained/custom models and regime estimation.
- Person 3: strategies, friction-aware decisions, portfolio optimization, risk and evaluation.

Respect exclusive file ownership. Do not modify another person's production-owned files without explicit contract change and

PHASE COMPLETION REPORT
After each phase, report:
1. What changed.
2. Exact files created or modified.
3. Tests executed and actual results.
4. Assumptions and unresolved issues.
5. Interface changes.
6. Remaining risks.
7. Exact next implementation step.

Begin with the repository audit only. Do not implement the entire system in one pass.
```

PART XIII: FINAL ACCEPTANCE CRITERIA

38. Definition of Done

- The required competition interface works end to end.
- The system processes timestamped data without future leakage.
- Quantitative strategies and accepted model predictions feed one decision engine.
- Every candidate trade is evaluated after applicable friction.
- Strategy selection adapts to market regime and estimated execution quality.
- The risk layer can veto infeasible trades.
- Long and short positions are handled correctly wherever supported.
- Realized and unrealized P&L reconcile with actual fills.
- Profit realization and capital recycling are evaluated against simpler policies.
- The system has reproducible chronological evaluation and ablation results.
- Three teammates can develop and test separate subsystems without conflicting ownership.
- All performance claims are backed by measured results.

Sentinel's defining contribution is the integrated decision process: it forecasts, evaluates whether the edge survives friction, determines whether the action improves the portfolio, and decides whether capital should remain invested or be realized and redeployed.